

IASI Quarto User Guide

Framework for Engineering Documentation with Quarto

IASI Project

Table of contents

1	IASI Quarto User Guide	4
2	Definiciones y conceptos	5
2.1	Definiciones y conceptos	5
3	Instalación	7
3.1	Requisitos	7
3.2	Instalar <code>iasi.quarto</code>	7
3.3	Comprobar la instalación	7
4	Tipos de publicación	9
4.1	Estrategias soportadas	9
4.2	Configuración de una publicación	9
4.3	Crear un Website	10
4.4	Crear un Book Regular	11
4.5	Crear un Book Structured	12
4.6	Crear un Book Direct	13
4.7	Elegir una estrategia	13
5	Formatos	14
5.1	Formatos disponibles	14
5.2	Configurar un formato	14
5.3	Construir un formato	15
5.4	Construcción predeterminada	15
6	Construir y publicar	16
6.1	Operaciones principales	16
6.1.1	Validar	16
6.1.2	Construir	16
6.1.3	Publicar	17
6.1.4	Liberar	18
6.2	Flujo recomendado	19
7	Multiproyectos	20
7.1	Trabajar con varios proyectos IASI	20
7.2	<code>path</code> define el ámbito	20

7.3	Selección por operación	21
7.4	Un único árbol de publicación	21
8	Problemas frecuentes	23
8.1	El proyecto no se reconoce	23
8.2	Un proyecto hijo no se descubre	23
8.3	Un formato no se genera	23
8.4	Quiero reconstruir aunque nada haya cambiado	24
8.5	<code>publish()</code> dice que no hay cambios	24
8.6	Quiero publicar lo que ya está construido	24
8.7	Quiero preparar los entregables locales	24
8.8	Quiero desplegar a un servicio externo	24

1 IASI Quarto User Guide

`iasi.quarto` ayuda a organizar, construir y publicar proyectos documentales basados en Quarto.

IASI Quarto nació de una necesidad concreta: gestionar distintos tipos de publicaciones dentro de una misma estructura documental, compartiendo reglas de construcción, validación, formatos, publicación y despliegue sin obligar a que todas las publicaciones tengan que organizarse de la misma manera.

IASI es su primer caso de uso y el entorno en el que el framework se desarrolla y se valida. Por eso su implementación actual responde a problemas reales del propio proyecto. Sin embargo, el modelo no está ligado a IASI ni pretende ser una solución exclusiva para su estructura documental.

La idea es que otras organizaciones puedan partir de una infraestructura de publicación ya resuelta y realizar únicamente el *tailoring* necesario para adaptarla a su contexto: tipos de publicación, perfiles, formatos, estilos, convenciones y reglas propias. El pipeline, los contratos y los mecanismos comunes no deberían tener que reinventarse para cada organización.

En otras palabras: gran parte del problema ya está resuelto; cada organización adapta la parte que le pertenece.

Esta guía explica **cómo utilizarlo**. No describe su arquitectura interna ni el funcionamiento de sus componentes privados.

El recorrido recomendado es sencillo:

1. conocer los términos que utiliza IASI Quarto;
2. instalar la herramienta;
3. elegir el tipo de publicación y su estrategia;
4. configurar los formatos de salida;
5. construir y publicar;
6. utilizar un multiproyecto cuando varias publicaciones comparten un mismo espacio de trabajo.

Si ya conoce IASI Quarto, puede ir directamente al capítulo que corresponda a la tarea que quiere realizar.

2 Definiciones y conceptos

2.1 Definiciones y conceptos

En este documento se utilizan los siguientes términos y conceptos. Las definiciones se presentan en orden alfabético para facilitar su consulta.

Término	Definición
Build	Operación que construye uno o varios formatos de una publicación a partir de sus fuentes. Por defecto, los resultados se escriben en <code>_outputs/</code> ; la ubicación puede configurarse. Véase: Formato.
Capítulo	Unidad principal de contenido de un libro. Puede estar organizado en secciones.
Direct	Estrategia de libro en la que cada documento <code>.qmd</code> del contenido se publica como capítulo y las carpetas se utilizan únicamente para organizar físicamente los archivos. Véase: Estrategia.
Deploy	Operación que mantiene una publicación al día: construye cuando es necesario y publica cuando el resultado construido ha cambiado. Puede ejecutarse en modo forzado. Véase: Force y Publish.
Estrategia	Regla que determina cómo interpreta IASI Quarto la organización física de un libro. Las estrategias de libro son regular, structured y direct.
Force	Modo explícito que ordena ejecutar la operación completa sin consultar si el estado previo indica que es necesaria. No evita las validaciones ni convierte operaciones Git en operaciones forzadas.
Formato	Tipo de salida que puede generar una publicación, por ejemplo HTML, PDF o EPUB.
Git-Book	Formato de publicación de IASI Quarto para generar un libro destinado a GitBook. Se selecciona mediante el formato <code>git</code> . Véase: Formato.
Libro	Publicación Quarto de tipo book. IASI Quarto admite las estrategias regular, structured y direct para libros. Véase: Publicación.
Multi-proyecto	Espacio de trabajo que contiene varias publicaciones IASI Quarto y permite operar sobre todas o seleccionar una de ellas. Véase: Publicación.

Término	Definición
Parte	División de alto nivel de un libro utilizada para agrupar capítulos relacionados. En la estrategia structured, una carpeta de contenido representa una Parte. Véase: Structured.
Proyecto Quarto	Proyecto reconocido por Quarto mediante su configuración habitual, incluida <code>__quarto.yml</code> .
Publicación	Proyecto Quarto gestionado por IASI Quarto mediante <code>__iasi.yml</code> . Puede ser un Website o un Libro. Véase: Website.
Publish	Operación que toma los resultados ya construidos y crea el árbol final publish/preparado para desplegarse. No vuelve a renderizar los fuentes.
Quarto	Motor de publicación que procesa los documentos <code>.qmd</code> y genera los formatos finales. IASI Quarto organiza y coordina el proyecto, pero Quarto continúa siendo el motor de renderizado.
Regular	Estrategia de libro en la que cada carpeta de contenido se convierte en una unidad o capítulo y los documentos <code>.qmd</code> que contiene se combinan en el <code>index.qmd</code> generado de esa carpeta. Véase: Estrategia.
Sección	División principal dentro de un capítulo.
Structured	Estrategia de libro en la que cada carpeta de contenido representa una Parte, su <code>index.qmd</code> presenta esa Parte y los demás documentos <code>.qmd</code> de la carpeta son Capítulos. Véase: Estrategia.
Website	Publicación Quarto de tipo website. En IASI Quarto utiliza la estrategia regular.

3 Instalación

3.1 Requisitos

Para utilizar `iasi.quarto` necesita:

- R;
- Quarto;
- acceso a los formatos adicionales que quiera generar, por ejemplo una instalación TeX para PDF;
- Git si va a instalar el paquete directamente desde GitHub.

Puede comprobar R y Quarto desde una terminal:

```
R --version
quarto --version
```

3.2 Instalar `iasi.quarto`

Una forma sencilla de instalar la versión disponible en GitHub es utilizar `remotes`:

```
install.packages("remotes")
remotes::install_github("iasi-org/iasi-quarto")
```

Después puede cargar el paquete:

```
library(iasi.quarto)
```

3.3 Comprobar la instalación

Compruebe la versión instalada:

```
packageVersion("iasi.quarto")
```

Y, desde una publicación IASI Quarto, ejecute:

```
validate()
```

Si el proyecto es válido, IASI Quarto lo reconocerá sin construir todavía ningún formato.

4 Tipos de publicación

4.1 Estrategias soportadas

IASI Quarto soporta tres estrategias de publicación. La estrategia define cómo se interpreta la organización de los documentos fuente.

Es- trate- gia	Disponible para	Organización editorial	Cuándo utilizarla
reg- ular	Web- site y Book	En un Book, cada carpeta de contenido forma una unidad y sus archivos .qmd se combinan.	Cuando varios documentos fuente forman una única unidad editorial. Es la estrategia predeterminada.
struc- tured	Book	Cada carpeta representa una Parte; su index.qmd presenta la Parte y los demás archivos .qmd son Capítulos.	Cuando la estructura Parte → Capítulo debe estar representada explícitamente.
di- rect	Book	Cada archivo .qmd se publica directamente como Capítulo; las carpetas solo organizan los fuentes.	Cuando cada documento ya es una unidad editorial completa.

Para un Website, la estrategia soportada es **regular**. Las estrategias **structured** y **direct** son propias de publicaciones de tipo Book.

4.2 Configuración de una publicación

Una publicación IASI Quarto combina la configuración de Quarto con `_iasi.yml`.

`_quarto.yml` continúa describiendo el proyecto Quarto. `_iasi.yml` indica cómo debe interpretar IASI Quarto su organización.

Una configuración mínima de libro puede ser:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Los valores predeterminados para un libro son:

```
strategy    = regular
content-dir = chapters
numbered    = true
```

`content-dir` indica dónde se encuentra el contenido organizado de la publicación.

Cuando `numbered: true`, los documentos y carpetas de contenido que participan en el orden editorial utilizan prefijos numéricos, por ejemplo:

```
chapters/
  01-intro/
  02-installation/
  03-usage/
```

4.3 Crear un Website

Un Website utiliza el tipo de proyecto `website` de Quarto y la estrategia `regular` de IASI Quarto.

En `_quarto.yml`:

```
project:
  type: website
```

En `_iasi.yml` puede utilizar:

```
publication:
  strategy: regular
```

Para un Website, IASI Quarto no introduce las estrategias editoriales propias de los libros. La estructura y navegación continúan siendo las del proyecto Website de Quarto.

Para construirlo:

```
build(format = "html")
```

4.4 Crear un Book Regular

`regular` es la estrategia predeterminada para libros.

La idea es:

```
carpeta de contenido
  → capítulo o unidad de publicación

documentos .qmd dentro de la carpeta
  → contenido combinado de esa unidad
```

Ejemplo:

```
chapters/
  01-intro/
    front-matter.quarto
    01-purpose.qmd
    02-scope.qmd
```

IASI Quarto genera el `index.qmd` de la carpeta combinando los documentos fuente.

`front-matter.quarto` puede proporcionar el front matter del `index.qmd` generado:

```
---
title: "Introducción"
title-block-style: none
---
```

Por tanto, **no edite el `index.qmd` generado**. Edite los documentos fuente y, cuando corresponda, `front-matter.quarto`.

Use `regular` cuando una carpeta represente una unidad editorial formada por varios fragmentos o documentos fuente.

4.5 Crear un Book Structured

`structured` representa explícitamente Partes y Capítulos.

La regla es:

```
carpeta de contenido
  → Parte

index.qmd de la carpeta
  → presentación de la Parte

demás .qmd de la carpeta
  → Capítulos
```

Ejemplo:

```
chapters/
  20-methodologies/
    index.qmd
    01-history.qmd
    02-current.qmd
```

En `_iasi.yml`:

```
publication:
  strategy: structured
  content-dir: chapters
  numbered: true
```

A diferencia de `regular`, el `index.qmd` de cada carpeta es **fuentes** y debe existir. IASI Quarto no lo genera a partir de otros documentos.

`front-matter.quarto` no participa en la estrategia `structured`.

Use `structured` cuando quiera que la jerarquía editorial Parte → Capítulo esté representada directamente por la estructura del proyecto.

4.6 Crear un Book Direct

`direct` publica cada documento `.qmd` como capítulo.

Las carpetas sirven para ordenar físicamente los fuentes, pero no crean Partes ni capítulos contenedores.

Ejemplo:

```
chapters/  
  01-concepts/  
    01-model.qmd  
    02-context.qmd  
  02-usage/  
    01-build.qmd  
    02-publish.qmd
```

La publicación resultante contiene los cuatro documentos como capítulos consecutivos.

En `_iasi.yml`:

```
publication:  
  strategy: direct  
  content-dir: chapters  
  numbered: true
```

Use `direct` cuando cada `.qmd` ya sea una unidad editorial completa y las carpetas solo deban ayudar a mantener organizado el repositorio.

4.7 Elegir una estrategia

Si necesita...	Estrategia
Un Website Quarto	regular
Combinar varios archivos <code>.qmd</code> de una carpeta en una única unidad	regular
Representar Partes que contienen Capítulos	structured
Publicar cada archivo <code>.qmd</code> directamente como Capítulo	direct

La elección es editorial. No depende del formato final: un mismo libro puede utilizar la misma estrategia para HTML, PDF, EPUB y otros formatos.

5 Formatos

5.1 Formatos disponibles

IASI Quarto reconoce actualmente los siguientes formatos de construcción:

Formato	Resultado habitual
html	Publicación HTML
pdf	PDF generado por Quarto
typst	PDF mediante Typst
epub	Libro EPUB
doc	Documento ODT presentado como formato DOC de IASI Quarto
odt	Documento ODT
git	GitBook

Un Website utiliza HTML. Un Book puede utilizar cualquiera de los formatos soportados que tenga configurados.

El formato `git` representa una publicación **GitBook**. `gfm` no es un formato soportado por IASI Quarto.

5.2 Configurar un formato

Cada formato se configura mediante su perfil Quarto correspondiente, por ejemplo:

```
_quarto-html.yml  
_quarto-pdf.yml  
_quarto-epub.yml
```

IASI Quarto no sustituye la configuración propia de Quarto para cada formato.

5.3 Construir un formato

Para construir solo HTML:

```
build(format = "html")
```

Para construir solo PDF:

```
build(format = "pdf")
```

Puede solicitar varios formatos:

```
build(format = c("html", "pdf"))
```

5.4 Construcción predeterminada

Si no indica `format`, IASI Quarto utiliza su conjunto predeterminado:

```
html, pdf, epub, doc, git
```

Solo se construyen los formatos que estén disponibles en la publicación mediante sus perfiles correspondientes.

6 Construir y publicar

6.1 Operaciones principales

La API pública de `iasi.quarto` gira alrededor de cuatro operaciones:

```
validate()
build()
publish()
release()
```

Las cuatro utilizan la misma infraestructura de descubrimiento y selección. A partir de `path`, IASI recorre el árbol, descubre los proyectos IASI y selecciona únicamente los tipos a los que aplica la operación solicitada.

6.1.1 Validar

```
validate()
```

`validate()` comprueba los proyectos IASI seleccionados.

Para publicaciones Quarto valida su configuración y estructura. Para paquetes R comprueba, entre otras condiciones, la existencia de `DESCRIPTION`.

No construye, publica ni libera artefactos.

6.1.2 Construir

```
build()
```

`build()` construye siempre que se invoca. No intenta decidir si los fuentes han cambiado.

Para publicaciones Quarto, los resultados se materializan en `_outputs/`, separados por perfil/formato:


```
publication/  
  _outputs/  
    html/  
    pdf/  
    epub/  
    ...
```

Durante la edición puede construirse un formato concreto:

```
build(format = "html")
```

Si no se indica **format**, se construyen los perfiles declarados por el proyecto para los que exista `render`.

Para un proyecto **r-package**, `build()` genera los dos artefactos del paquete en su raíz:

```
package/  
  package_version.tar.gz  
  package_version.zip
```

El primero es el paquete fuente y el segundo el paquete binario en Windows.

6.1.3 Publicar

```
publish()
```

`publish()` trabaja únicamente con proyectos publicables. Toma siempre como entrada el árbol `_outputs/`; no vuelve a renderizar las fuentes.

Antes de publicar calcula una firma del contenido de `_outputs/`. Si coincide con la firma de la última publicación, la operación se omite. Puede forzarse una nueva publicación aunque el contenido no haya cambiado:

```
publish(force = TRUE)
```

La preparación se realiza sobre un árbol temporal **.work**. Solo cuando la copia y el postprocesado terminan correctamente se sustituye el destino final. Si el proceso falla, la publicación anterior permanece intacta.

En una publicación individual:

```
publication/  
  _outputs/  
  _publish/  
    index.html  
    chapters/  
    pdf/  
      libro.pdf  
    epub/  
      libro.epub
```

El contenido HTML ocupa la raíz del árbol publicable; los demás formatos permanecen como exports.

La metadata `.publish` conserva el timestamp y la firma del contenido que originó la publicación.

6.1.4 Liberar

```
release()
```

`release()` reúne los entregables locales. No despliega nada en servicios externos.

Para un `r-package`, mueve a `release/` los artefactos construidos (`*.tar.*` y `*.zip`).

Para proyectos publicables, copia a `release/` el contenido previamente materializado por `publish()`.

Por tanto, el ciclo queda separado deliberadamente:

```
validate  → comprobar  
build     → construir artefactos  
publish   → preparar publicaciones  
release   → reunir entregables
```

`iasi.quarto` termina aquí. El despliegue a sistemas externos pertenece a herramientas de nivel superior, como `iasi-dev`.

6.2 Flujo recomendado

Durante la edición, construya únicamente el formato que necesita revisar:

```
build(format = "html")
```

Cuando quiera materializar todos los perfiles declarados:

```
build()
```

Cuando el resultado construido sea publicable:

```
publish()
```

Una segunda llamada a `publish()` no repite trabajo si `_outputs/` no ha cambiado. Para repetir expresamente la publicación:

```
publish(force = TRUE)
```

Cuando quiera reunir los entregables locales:

```
release()
```

La regla práctica es:

```
build    → construye siempre  
publish  → publica si cambió; force publica siempre  
release  → recoge lo ya construido/publicado
```

7 Multiproyectos

7.1 Trabajar con varios proyectos IASI

Un repositorio puede contener varios proyectos IASI bajo una misma raíz. Los directorios intermedios no necesitan configuración IASI: el descubrimiento continúa descendiendo hasta encontrar un directorio que contenga `_iasi.yml` o `.iasi.yml`, salvo las ramas definidas como excluidas.

Ejemplo:

```
repository/  
  _iasi.yml  
  docs/  
    user-guide/  
      _iasi.yml  
      _quarto.yml  
      _outputs/  
  software/  
    iasi-quarto/  
      .iasi.yml  
      DESCRIPTION
```

En este ejemplo, `docs/` y `software/` son únicamente contenedores. Los proyectos IASI son la raíz `repository/`, `docs/user-guide/` y `software/iasi-quarto/`.

7.2 path define el ámbito

Las operaciones utilizan `path` como único selector del ámbito:

```
validate(path = ".")  
build(path = ".")  
publish(path = ".")  
release(path = ".")
```

Si se ejecuta una operación desde una subcarpeta, el descubrimiento se limita a ese árbol.

No existe un selector adicional `book`: para trabajar sobre un proyecto concreto, utilice su `path`.

Por ejemplo:

```
build(path = "docs/user-guide", format = "html")
```

7.3 Selección por operación

Descubrir un proyecto no significa necesariamente procesarlo.

Cada operación selecciona de los proyectos descubiertos únicamente los tipos a los que aplica:

```
validate → guide, web, r-package  
build    → guide, web, r-package  
publish  → guide, web  
release  → guide, web, r-package
```

Un proyecto de tipo `repository` puede servir como raíz de configuración y contener otros proyectos IASI, pero no se procesa como una publicación o un paquete.

7.4 Un único árbol de publicación

Cuando `publish()` se ejecuta desde la raíz que contiene varias publicaciones, utiliza un único `_publish/` común. Cada publicación ocupa un slot estable.

Los prefijos numéricos organizativos se eliminan al derivar el nombre del slot:

```
01-user-guide      → _publish/user-guide/  
02-developer-guide → _publish/developer-guide/
```

Cada publicación se prepara primero en su árbol temporal y sustituye únicamente su destino. Publicar `user-guide` no modifica `developer-guide`.

Cuando `path` apunta directamente a una publicación individual, esa publicación utiliza su propio `_publish/`.

Los niveles quedan separados:

fuentes

- `_outputs/` artefactos contruidos
- `_publish/` publicación local preparada
- `release/` entregables locales

8 Problemas frecuentes

8.1 El proyecto no se reconoce

Compruebe que se encuentra en la ruta correcta y que el proyecto contiene `_iasi.yml` o `.iasi.yml`.

Ejecute:

```
validate()
```

Para una publicación Quarto también debe existir `_quarto.yml`; para un `r-package`, `DESCRIPTION`.

8.2 Un proyecto hijo no se descubre

Los directorios intermedios no necesitan una marca IASI. El descubrimiento desciende por el árbol hasta encontrar proyectos, excepto dentro de ramas excluidas.

Compruebe que el directorio del proyecto contiene directamente `_iasi.yml` o `.iasi.yml` y que no está bajo una ruta excluida.

8.3 Un formato no se genera

Compruebe que el perfil está declarado por el proyecto y que existe el renderer correspondiente.

Por ejemplo:

```
_quarto-pdf.yml
```

También debe disponer de las dependencias que necesite el formato. PDF, por ejemplo, puede requerir una instalación TeX funcional.

8.4 Quiero reconstruir aunque nada haya cambiado

Utilice:

```
build()
```

`build()` construye siempre que se invoca.

8.5 `publish()` dice que no hay cambios

`publish()` calcula una firma de `_outputs/` y la compara con la guardada en la última publicación.

Si quiere repetir expresamente el proceso:

```
publish(force = TRUE)
```

8.6 Quiero publicar lo que ya está construido

Utilice:

```
publish()
```

`publish()` toma siempre su entrada de `_outputs/`.

8.7 Quiero preparar los entregables locales

Utilice:

```
release()
```

Para paquetes R recoge los artefactos fuente y binario. Para publicaciones recoge el contenido previamente preparado por `publish()`.

8.8 Quiero desplegar a un servicio externo

El despliegue externo no forma parte de `iasi.quarto`. Utilice la herramienta responsable del entorno de desarrollo o despliegue, por ejemplo `iasi-dev`.